

Complete Matrix DSA Notes: Basic to Advanced

1. Matrix Basics

Matrix is a 2D array.

Element:

$A[i][j]$

Rows = m

Columns = n

Most matrix problems depend on:

- traversal
- transformations
- searching
- DFS/BFS
- Dynamic Programming
- mathematical operations

2. Matrix Representation

Python:

```
matrix = [[1,2,3],[4,5,6]]
```

C++:

```
vector<vector<int>> matrix;
```

Java:

```
int[][] matrix;
```

3. Row Traversal

Visit every element row by row.

Time Complexity:

$O(m*n)$

Space:

$O(1)$

4. Column Traversal

Outer loop over columns.
Inner loop over rows.

Used in:
- transpose
- column calculations

5. Diagonal Traversal

Main diagonal condition:

$$i == j$$

Anti diagonal:

$$i + j == n-1$$

6. Matrix Addition

Formula:

$$C[i][j] = A[i][j] + B[i][j]$$

Time:
 $O(m*n)$

7. Matrix Multiplication

For:

$$A = m*n$$
$$B = n*p$$

Result:
 $m*p$

Formula:

$$C[i][j] += A[i][k] * B[k][j]$$

Normal complexity:
 $O(n^3)$

8. Transpose Matrix

Rows become columns.

Formula:

$$B[j][i] = A[i][j]$$

Square matrix can be transposed in-place.

9. Rotate Matrix 90 Degrees

Algorithm:

1. Transpose matrix
2. Reverse every row

Complexity:
 $O(n^2)$

Space:
 $O(1)$

10. Spiral Matrix

Boundary method:

Maintain:

top
bottom
left
right

Move:
right
down
left
up

Common interview pattern.

11. Binary Search Matrix

If matrix is fully sorted:

Convert 2D into imaginary 1D.

row = mid / cols
col = mid % cols

Complexity:
 $O(\log(m*n))$

12. Row Column Sorted Search

Start from top right.

If current > target:
move left

If current < target:
move down

Complexity:
 $O(m+n)$

13. Set Matrix Zeroes

Goal:
If a cell is zero,
make complete row and column zero.

Optimal:

Use first row and first column as markers.

Space:
 $O(1)$

14. Flood Fill DFS

DFS explores connected cells.

Directions:

(1,0)
(-1,0)
(0,1)
(0,-1)

Applications:

- islands
- images
- connected components

15. BFS Matrix

Uses Queue.

Best for shortest distance.

Applications:

- Rotting oranges
- shortest path
- nearest cell

16. Prefix Sum Matrix

Allows fast rectangle queries.

Formula:

$$\begin{aligned} \text{prefix}[i][j] = & \\ & \text{grid}[i][j] \\ & + \text{top} \\ & + \text{left} \\ & - \text{diagonal} \end{aligned}$$

Query:

$O(1)$

17. Matrix Dynamic Programming

State:

$\text{dp}[i][j]$

Examples:

Unique Paths:

$\text{dp}[i][j] =$

$dp[i-1][j] + dp[i][j-1]$

Minimum Path Sum:

$grid[i][j] + \min(up, left)$

18. Backtracking Matrix

Used in:

- Sudoku Solver
- Word Search
- Rat Maze

Pattern:

choose
explore
undo

19. Matrix As Graph

Every cell is considered a graph node.

Cell:
(i,j)

Neighbors:
4-direction or 8-direction

Algorithms:

DFS
BFS
Dijkstra
Union Find

Problems:

Number of Islands
Maze
Pacific Atlantic Water Flow

20. Advanced Matrix Algorithms

Gaussian Elimination:
Solve equations.
 $O(n^3)$

Matrix Exponentiation:
Fast recurrence solving.

Power calculation:
 $O(\log n)$

Strassen Multiplication:

Improves matrix multiplication.

 $O(n^{2.807})$

Python Core Matrix Templates

```
# Traversal
for i in range(rows):
    for j in range(cols):
        print(matrix[i][j])

# Directions
dirs=[
    (1,0),
    (-1,0),
    (0,1),
    (0,-1)
]

# DFS
def dfs(i,j):
    if invalid:
        return

    visited.add((i,j))

    for dx,dy in dirs:
        dfs(i+dx,j+dy)
```

C++ Core Template

```
vector<pair<int,int>> dirs={
    {1,0},
    {-1,0},
    {0,1},
    {0,-1}
};
void dfs(int i,int j){
```

```

for(auto d:dirs){
    int ni=i+d.first;
    int nj=j+d.second;

}
}

```

Java Core Template

```

int[][] dirs={
    {1,0},
    {-1,0},
    {0,1},
    {0,-1}
};

void dfs(int i,int j){
    for(int[] d:dirs){
        int ni=i+d[0];
        int nj=j+d[1];

    }
}

```

Matrix Problem Roadmap

Easy:

- Flood Fill
- Toeplitz Matrix
- Matrix Diagonal Sum

Medium:

- Spiral Matrix
- Rotate Image
- Set Matrix Zeroes
- Number of Islands
- Word Search

Hard:

- Sudoku Solver
- Longest Increasing Path
- Trapping Rain Water II
- Cherry Pickup
- Maximal Rectangle